

RetinaXAI

Arhitectura conceptuală a platformei

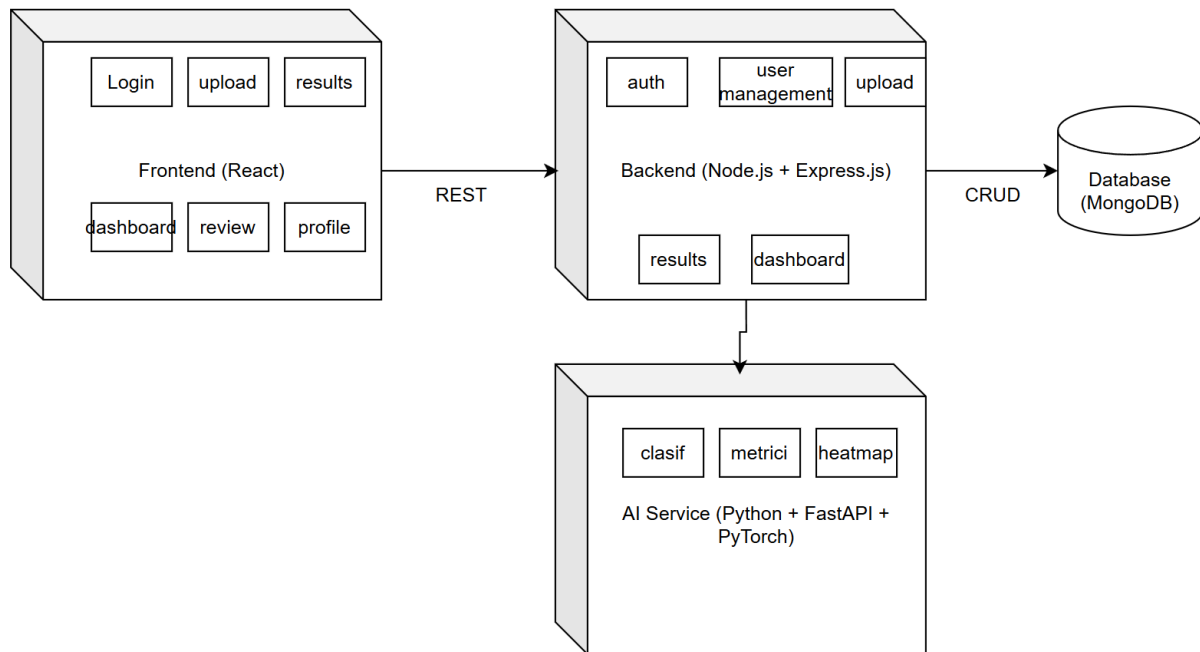
Arhitectura conceptuală a platformei RetinaXAI este bazată pe un model de tip Client–Server, cu un serviciu separat pentru procesarea modelului de inteligență artificială.

1. Modulul Frontend reprezintă interfața cu utilizatorul și include paginile principale ale aplicației, precum pagina Home, autentificare, încărcarea imaginilor, vizualizarea rezultatelor și dashboard-ul. Prin intermediul acestuia, utilizatorul poate încărca imagini de retină și poate vizualiza predicțiile și explicațiile generate.
2. Modulul Backend gestionează logica aplicației și comunicarea dintre frontend, baza de date și serviciul AI. Acesta se ocupă de autentificare, gestionarea rolurilor utilizatorilor și salvarea rezultatelor în baza de date.
3. Modulul AI Engine este responsabil de procesarea imaginilor și de generarea predicțiilor. În cadrul acestuia sunt calculate predicția finală, harta de explainability folosind Grad-CAM, precum și metricile de evaluare precum Fidelity, Stability și Human Agreement. În cazul rolului de doctor, rezultatele pot fi ulterior validate și comparate cu observațiile expertului, contribuind la calculul metricii Human Agreement.
4. Baza de date stochează utilizatorii, imaginile încărcate, istoricul analizelor și rezultatele generat si observațiile/adnotările realizate de doctori.

Fluxul principal este următorul: utilizatorul încarcă imaginea prin interfața web, cererea este trimisă către backend, care o transmite către modulul AI pentru procesare, iar rezultatele sunt salvate în baza de date și afișate în aplicație.

Vor exista si rolurile:

Rol	Permisiuni
Utilizator	Încarcă imagini de retină, vizualizează rezultatele, consultă istoricul analizelor și dashboard-ul personal
Doctor	Vizualizează cazurile pacienților, adaugă observații medicale, realizează adnotări și validează explicațiile modelului pentru evaluarea Human Agreement
Administrator	Gestionează utilizatorii, monitorizează analizele și vizualizează statisticile globale ale platformei



Arhitectura tehnică a platformei

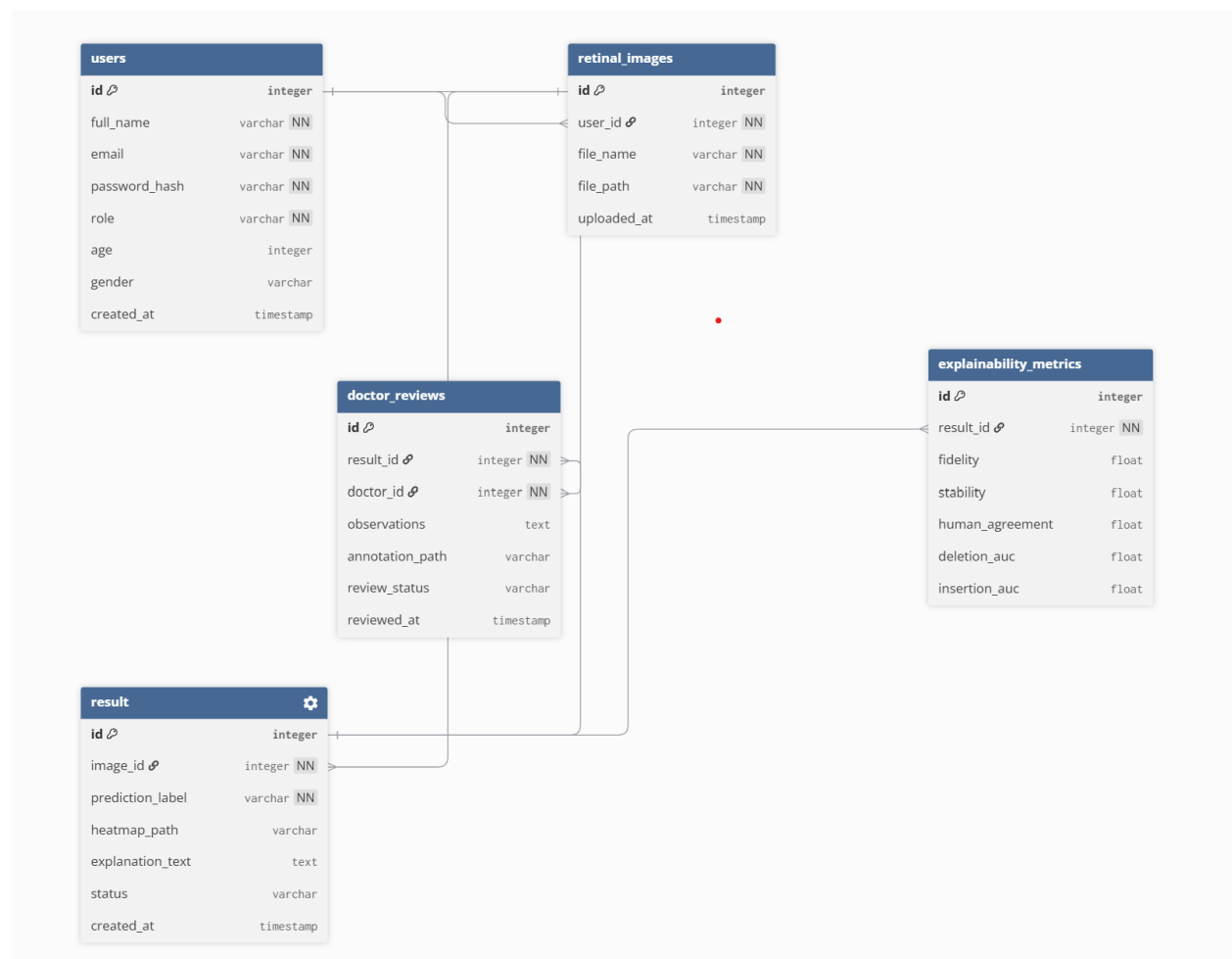
Arhitectura tehnică a platformei RetinaXAI este bazată pe infrastructura MERN:

1. Frontend: React – ales pentru realizarea unei interfețe web moderne și interactive, potrivită pentru pagini precum autentificare, upload imagine, rezultate, dashboard și pagina de review pentru doctor.
2. Backend: Node.js + Express.js – utilizat pentru logica aplicației, autentificare, gestionarea utilizatorilor și comunicarea cu baza de date și modulul AI.
3. Baza de date: MongoDB – folosită pentru stocarea utilizatorilor, imaginilor încărcate, rezultatelor analizelor, observațiilor medicilor și metricilor calculate. A fost aleasă deoarece oferă flexibilitate în gestionarea datelor și se integrează foarte bine în infrastructura MERN.
4. Legătura cu serviciul AI: Python + FastAPI – modelul de clasificare, generarea hărților Grad-CAM și calculul metricilor sunt realizate în Python, iar FastAPI este utilizat pentru expunerea endpoint-urilor necesare comunicării cu backend-ul. Se va folosi PyTorch pentru modelul de clasificare și generarea explainability maps prin Grad-CAM.

Comunicarea dintre frontend și backend, respectiv dintre backend și modulul AI, se va realiza prin REST API, folosind protocolul HTTP și schimb de date în format JSON, deoarece această

metodă este standard în aplicațiile web și permite o comunicare simplă și eficientă între componente.

Modelul de date



Interfețele de comunicare dintre frontend și backend

1. Autentificare

POST /api/auth/register

Creează un cont nou de utilizator.

Request body:

- full_name: string
- email: string
- password: string
- age: integer
- gender: string
- role: string

Response:

- message: string
- userId: integer

POST /api/auth/login

Autentifică utilizatorul și returnează datele necesare sesiunii.

Request body:

- email: string
- password: string

Response:

- token: string
- id: integer
- full_name: string
- role: string

2. Profil utilizator

GET /api/profile

Returnează datele profilului utilizatorului.

Id-ul utilizatorului este trimis în header prin token.

Response:

- id: integer
- full_name: string
- email: string
- age: integer
- gender: string
- role: string

PUT /api/profile

Actualizează informațiile utilizatorului, precum numele, vârsta sau genul.

Request body:

- full_name: string
- age: integer
- gender: string

Response:

- message: string
- updatedUser: object

3. Încărcare imagine și analiză

POST /api/upload

Încarcă imaginea de rețină și pornește procesarea AI.

Request body:

- image: file

Response:

- analysisId: integer
- prediction_label: string
- confidence: float

Acest endpoint este utilizat în pagina /upload.

4. Vizualizare rezultate

GET /api/analyses/{id}

Returnează rezultatul complet al unei analize.

Response:

- analysisId: integer
- prediction_label: string
- confidence: float
- explanation_text: string

GET /api/analyses/{id}/metrics

Returnează metricile de explainability asociate.

Response:

- fidelity: float
- stability: float
- human_agreement: float
- deletion_auc: float
- insertion_auc: float

GET /api/analyses/{id}/heatmap

Returnează harta Grad-CAM generată.

Response:

- heatmap_path: string

Aceste endpoint-uri sunt utilizate în pagina /results.

5. Dashboard utilizator

GET /api/dashboard

Returnează statisticile personale ale utilizatorului.
Id-ul este identificat din token.

Response:

- total_images: integer
- total_analyses: integer
- average_fidelity: float
- average_stability: float

GET /api/history

Returnează istoricul analizelor realizate de utilizator.

Response:

- history: array<object>

Fiecare obiect din array conține:

- analysisId: integer
- prediction_label: string
- date: string

Aceste endpoint-uri sunt utilizate în pagina /dashboard.

6. Review doctor

GET /api/reviews/{analysisId}

Returnează analiza care urmează să fie revizuită de doctor.

Response:

- analysisId: integer
- prediction_label: string
- confidence: float
- heatmap_path: string

POST /api/reviews

Salvează observațiile, adnotările și decizia doctorului.

Request body:

- analysisId: integer
- observations: string
- annotation_path: string
- review_action: string

Response:

- reviewId: integer
- message: string

Aceste endpoint-uri sunt utilizate în pagina /review.

7. Dashboard administrator

GET /api/admin/dashboard

Returnează statisticile globale ale platformei.

Response:

- total_users: integer
- total_analyses: integer
- average_fidelity_global: float
- average_stability_global: float
- average_human_agreement_global: float

GET /api/admin/users

Returnează lista utilizatorilor.

Response:

- users: array<object>

GET /api/admin/analyses

Returnează toate analizele existente în sistem.

Response:

- analyses: array<object>

Aceste endpoint-uri sunt utilizate în pagina /dashboardAdmin.